UAIX Lab-Atlas

Setup and first-run guide

Version	1.0, 14 October 2026
For	A designer who has never used these skills
You will end with	30 skills installed, one practice project, and one reviewed design artifact
Time needed	About 60 minutes, plus account setup if you have none of the accounts below
Verified	macOS with Claude Code: installer checked. Everything marked UNVERIFIED has not been tested yet.
Licence	None yet. Pilot use only. Do not redistribute.

Contents

1. What this is

2. What you need before you start

3. Safety rules (read once)

4. Get the files

5. Set up Claude

6. Install the skills

7. Check that it worked

8. Connectors and MCP servers

9. First run: plan, execute, audit

10. Do you have a design system?

11. If something goes wrong

12. Tell us what happened

Appendix A. The 30 skills

Appendix B. Every link in this guide

Appendix C. Words used in this guide

1. What this is

UAIX Lab-Atlas is a set of 30 skills for Claude. A skill is a folder of instructions that teaches Claude one design job, such as writing a product brief, analysing interview transcripts, building a journey map or auditing a screen. Each skill asks you for context first, shows you a plan, tags every claim with its evidence, and ends by asking you to review.

This guide takes you from the GitHub link to your first reviewed artifact. Follow it in order. Every step that can go wrong has a **Check** line that tells you whether it is safe to continue.

The work lives in the GitHub repository RPalis/ux-atlas-aixlab, on the branch `update-readme-retire-competitors-duplicate`.

> **NOTE**
> This guide has no screenshots. Button names change when Claude, GitHub and Figma update their screens. If a name here does not match yours, look for the closest one, and write the difference down in the feedback form (section 12). That is a defect in the guide, not a mistake by you.

2. What you need before you start

Set these up first. The last column says which ones you can skip.

Item	Used for	Link	Required?
GitHub account (free)	Opening the repository	github.com/signup	Yes, for routes A and B. Not for route C.
Claude account with a paid plan (Pro, Max, Team or Enterprise), or a Claude Console account	Claude Code needs one of these to sign in	claude.com/pricing	Yes for Track 1 (Claude Code)
Claude account, any plan, with code execution switched on	Uploading skills in the browser (Track 2)	claude.ai	Yes for Track 2 only
Claude desktop app (Mac or Windows)	Claude Code with its Code tab	claude.com/download	Yes for Track 1, unless you use a terminal
A terminal	Running the installer	Mac: the Terminal app. Windows: see section 6.	Yes for Track 1

Item	Used for	Link	Required?
Git for Windows or WSL	A Linux-style shell on Windows	git-scm.com/downloads/win and learn.microsoft.com/windows/wsl/install	Windows only (UNVERIFIED)
Node.js (LTS)	Skills that build or package a prototype: `prototype`, `storybook`, `package`, `showcase`	nodejs.org	Not for your first run
Figma account	Skills that move work to and from Figma	figma.com	Not for your first run
An empty folder called `atlas-practice`	Your practice project	Create it on your Desktop	Yes

Time: about 60 minutes for the guide itself. Account setup can take longer.

> **NOTE**
> Never use client material. Your practice project must use an invented product idea. For example: an app to share weekly meal plans with housemates.

3. Safety rules (read once)

These seven rules apply to every step. They exist because a skill can write files, and a first session must not be able to damage real work or invent research.

Rule	What to do	Why
G1 Scope	Work only inside the folder `atlas-practice`. Never point a skill at a client project, a production repository or a folder with real research.	A skill can write files.
G2 Secrets	Never paste passwords, API keys, tokens, client names or real user data into any chat. Sign in to services in your browser only.	Claude never needs them for this guide.
G3 Permissions	Keep Claude on its default setting, which asks before editing. Do not switch on bypass, auto-accept or "dangerously skip permissions".	You approve every file change yourself.
G4 Plan first	A skill must show you a plan and wait before it writes a file. If it does not, type: stop, show me your plan first.	This is the main safety net.
G5 No invented evidence	A persona, quote, number or competitor fact must trace to a source you gave or that the skill found and cited. Otherwise it must say "unknown". Treat any untagged claim as invented.	Fabricated research is the failure these skills are built to prevent.

Rule	What to do	Why
G6 Human review	Read each output before you run the next skill.	Skills draft. You decide.
G7 Stop rule	Stop and tell the observer if a file appears outside `atlas-practice`, if Claude asks for a password or token, or if you are stuck for more than 5 minutes.	These are the three ways a session goes wrong.

4. Get the files

Your observer will tell you which route applies to you. Do only that route.

Route A: you were invited to the private repository

1. Open your email and find the message **RPalis invited you to collaborate on RPalis/ux-atlas-aixlab**. Click **View invitation**.
2. If GitHub asks you to sign in, sign in with the account the invitation was sent to.
3. Click **Accept invitation**.
4. You land on the repository page. Click the branch button, which says `main`, and choose `update-readme-retire-competitors-duplicate`.

> **CHECK**
> You see a file list that includes `skills`, `README.md`, `install.sh` and `ONBOARDING.md`. If you see "404 Not Found", you are signed in to the wrong GitHub account. Click your picture at the top right, check the username, and switch account.

Shortcut: while signed in as the invited account, this link downloads the branch as a ZIP straight away:

```
https://github.com/RPalis/ux-atlas-aixlab/archive/refs/heads/update-readme-retire-competitors-duplicate.zip
```

Route B: the repository is public

1. Open https://github.com/RPalis/ux-atlas-aixlab.
2. Click the branch button, which says `main`, and choose `update-readme-retire-competitors-duplicate`.

> **CHECK**
> The file list includes `ONBOARDING.md`.

Route C: you were sent a ZIP file

1. Save the ZIP somewhere you can find it. Downloads is fine.
2. Unzip it. On a Mac, double-click it. On Windows, right-click it and choose **Extract All**.

Download the files (routes A and B)

1. On the repository page, with the branch `update-readme-retire-competitors-duplicate` selected, click the green **Code** button above the file list.
2. Click **Download ZIP**.
3. Open the downloaded file from your Downloads folder and unzip it.

> **CHECK**
> The unzipped folder contains `skills`, `README.md` and `install.sh`.

> **NOTE**
> If you already use Git you can run `git clone` instead. Most people should use the ZIP. A clone checks out `main`, so switch to the branch `update-readme-retire-competitors-duplicate` before you continue.

5. Set up Claude

Choose one track. Do only that track.

	Track 1: Claude Code	Track 2: Claude.ai in the browser
What you do	Install one app, run one installer, then work in a project folder	Upload skills one at a time in settings
Files	Skills read and write files in `atlas-practice`	No file access. Output comes back in the chat.
Plan-first gate	Claude Code plan mode and each skill's own plan step	Each skill's own plan step only
Best for	The full guide	A quick try of the research and thinking skills
Status	macOS checked	UNVERIFIED

Track 1: Claude Code

Option 1: the Claude desktop app (recommended)

1. Open claude.com/download.

2. Under the download options, click the one for your computer: macOS, or Windows (x64 or arm64).
3. Open the downloaded file and install the app. On a Mac, drag Claude into the Applications folder.
4. Open Claude and sign in with the account that has the paid plan.
5. Open the **Code** tab.

> **CHECK**
> The Code tab opens and lets you choose a working folder.

Option 2: Claude Code in a terminal

Use this if you prefer a terminal. Official instructions: code.claude.com/docs/en/quickstart. New to terminals? Start with code.claude.com/docs/en/terminal-guide.

1. Open a terminal. On a Mac, press Cmd+Space, type Terminal, press Enter.
2. Paste this command and press Enter. It shows no progress while it downloads.

```
curl -fsSL https://claude.ai/install.sh | bash
```

1. Open a new terminal window and run:

```
claude --version
```

> **CHECK**
> It prints a version number followed by (Claude Code). If your terminal says claude is not found, close it, open a new one and try again.

On Windows, run this in PowerShell instead of the command above (UNVERIFIED for this guide, but taken from the official quickstart):

```
irm https://claude.ai/install.ps1 | iex
```

Track 2: Claude.ai in the browser (UNVERIFIED)

Skills are uploaded as ZIP files, one folder per skill. This needs code execution switched on in your account. Official help: Using Skills in Claude.

1. Open claude.ai and sign in.
2. Open your settings and find **Capabilities**. Make sure **Code execution and file creation** is switched on.
3. Choose three skills to start with: `help`, `brief` and `interview-analysis`.

4. In the unzipped files, open the `skills` folder. For each of the three, make a ZIP of the skill folder itself. Mac: right-click the folder, then **Compress**. Windows: right-click, then **Send to**, then **Compressed (zipped) folder**.

> **CHECK**
> Opening each ZIP shows one folder with a file called `SKILL.md` inside it. Not loose files.

1. In Claude, open **Customize** then **Skills**. Older versions call this **Settings** then **Capabilities**. Use whichever your screen shows.
2. Click the upload button, choose the first ZIP, and confirm. Repeat for the other two.

> **CHECK**
> Each skill appears in the list and is switched on.

> **NOTE**
> Skills that write a documentation folder into a project (`setup` , `prototype` , `package`) cannot do that in chat. In Track 2 use the research and thinking skills: `brief` , `interview-analysis` , `problem-statement` , `brainstorming` , `information-architecture` , `ux-benchmark` , `usability-test-plan` , `persona` and `journey` .

6. Install the skills

Track 2 users: you already uploaded your skills in section 5. Skip to section 7.

Mac (checked)

1. Open **Terminal**: press Cmd+Space, type Terminal, press Enter.
2. Type `cd ` with a space after it. Drag the unzipped folder from Finder onto the Terminal window. Press Enter.
3. Run the installer:

```
./install.sh --claude --no-open
```

1. If you see "permission denied", run the command below, then run the installer again.

```
chmod +x install.sh
```

> **CHECK**
> The last lines say "Symlinked 30 skills into Claude Code" and "Successfully installed 30 skills!". If the number is not 30, stop and tell the observer.

What the installer did: it copied the skills to `~/.agents/skills` and linked each one into `~/.claude/skills`. It changed nothing else. The file `~/.agents/skills/.ux-atlas.manifest` lists exactly what it installed, so you can remove it later.

Windows (UNVERIFIED)

The installer is a bash script that makes symbolic links, so it needs a Linux-style shell. Choose one option and tell the observer which you used.

Option 1: WSL (recommended).

1. Open the Microsoft Store and install **Ubuntu**. Official help: learn.microsoft.com/windows/wsl/install.
2. Open Ubuntu, unzip the files inside it, and follow the Mac steps above. Claude Code must be installed in the same environment.

Option 2: copy the folders by hand, with no installer.

1. Open the unzipped `skills` folder.
2. Select all 30 folders and copy them into `C:\Users\<your name>\.claude\skills`. Create the `.claude\skills` folder first if it does not exist.

> **CHECK**
> Each folder inside `.claude\skills` contains a file called `SKILL.md`.

7. Check that it worked

Track 1

1. Create an empty folder called `atlas-practice`.
2. Open it in Claude Code. In the desktop app, open the **Code** tab and choose `atlas-practice` as the working folder. The button label may differ. In a terminal, run `cd atlas-practice` and then `claude`.
3. Turn on plan mode. Plan mode makes Claude show a plan and wait for you before it changes anything. In a terminal, press **Shift+Tab** until the indicator says plan mode. In the desktop app, use the mode selector near the prompt box, or Shift+Tab.
4. Type `/` and look at the list.

> **CHECK**
> The list includes skills such as `setup`, `brief`, `persona` and `audit`.

1. Type this exactly and press Enter:

```
Use the UAIX Lab-Atlas help skill and tell me which skill to run first.
```

> **CHECK**
> Claude names a skill and gives a reason tied to what it found. For an empty folder it should name `setup`.

> **NOTE**
> If you typed /help and saw Claude Code's own help screen, that is normal. The built-in command wins. Use the sentence above instead. If the skills do not appear, quit Claude fully, reopen it, and try again. Still nothing: see section 11.

Track 2

1. Start a new chat and type:

```
Use the help skill. I am starting a new product idea and have no files yet.
```

> **CHECK**
> Claude names a next skill and says why. If it answers without mentioning a skill, open your skills list and confirm `help` is switched on.

8. Connectors and MCP servers

An MCP server (also called a connector) lets Claude use another tool, such as Figma, on your behalf. **You do not need any of them for your first run.** Skip this section until a skill tells you it needs one. Nothing in sections 9 and 10 depends on it.

Tool	Skills that use it	Required?	Cost or limit
Figma MCP server (remote)	`figma` import and sync; `workshop` FigJam board; `map` FigJam export	Only for those modes. `figma` capture needs no MCP.	The remote server is available on all Figma seats and plans.

Tool	Skills that use it	Required?	Cost or limit
Figma MCP server (desktop)	Same	Optional alternative	Needs a Dev or Full seat on a paid Figma plan, and the Figma desktop app.
Jira or Azure DevOps MCP server	`requirements` export of epics and features	Optional. The doc is written without it.	Set up from your provider's own instructions. Not covered here.
Web search and page fetching	`competitor-analysis`, `ux-benchmark`	Needed to research competitors	Claude Code normally includes them. Availability can depend on plan and region.
Browser tooling (Playwright, or Claude in Chrome)	Real screenshots in `competitor-analysis` and `showcase`	Optional. Skills tell you if screenshots are skipped.	Playwright needs Node.js.
Output-format helper skills (docx, xlsx, pptx, humanizer)	Several research skills offer Word, Excel or PowerPoint output	Optional. Used only if installed.	Not part of this repository.

Set up the Figma MCP server (only when you need it)

Official guide: Guide to the Figma MCP server. The remote server address is `https://mcp.figma.com/mcp`. The command below uses the standard form from the Claude Code MCP documentation with that address. The Figma-specific command itself has not been tested for this guide, so tell the observer if it fails.

1. You need a Figma account. Open figma.com, sign in, and keep that browser window open.
2. Open a terminal. It does not need to be inside `atlas-practice`.
3. Run:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

1. Start Claude Code in `atlas-practice` (desktop app Code tab, or `claude` in a terminal).
2. Type `/mcp` and press Enter. Select **figma**.
3. Your browser opens. Sign in to Figma if asked, then click the button that allows access. The sign-in happens in the browser. You never type a Figma password or token into Claude.

> **CHECK**
> Back in Claude Code, `/mcp` shows figma as connected. If it shows failed, run `/mcp` again and repeat the browser sign-in.

> **STOP IF**
> Never create a Figma personal access token for this guide, and never paste one into a chat. If a skill or a web page asks you for one, stop and tell the observer (rule G2).

To see or remove servers later, run `claude mcp list` or `claude mcp remove figma`.

> **NOTE**
> Using the Claude desktop app instead of a terminal? Look in the app's settings for **Connectors** (the name may differ) and add Figma there. UNVERIFIED.

Node.js (only when you build a prototype)

1. Open nodejs.org and download the **LTS** version for your computer.
2. Run the installer and accept the defaults.
3. Open a new terminal and run:

```
node --version
```

> **CHECK**
> It prints a version number.

9. First run: plan, execute, audit

Use an invented product idea. Do not use a real client idea. All of this happens in `atlas-practice`.

Step 1: Plan

1. Track 1, in plan mode, type the line below with your own idea. Track 2: run `brief` instead and paste your one-sentence idea.

```
Use the setup skill. Project: <your idea in one sentence>. Starting point: brand-new idea.
```

1. Read the plan Claude shows.

> **STOP IF**
> If the plan mentions any file or folder outside `atlas-practice`, stop and tell the observer (rule G7).

1. Approve the plan. Then approve each file change one at a time (rule G3).

> **CHECK**
> Track 1: `atlas-practice` now contains `README.md`, `AGENTS.md`, and a folder named `<something>-docs` with a file `UNKNOWNS.md` inside it.

Step 2: Execute one skill

1. Run `brief`. It interviews you one question at a time.
2. Answer honestly. Say "I don't know" wherever you do not know.

> **CHECK**
> Every "I don't know" ends up in `UNKNOWNS.md` (Track 2: in an Unknowns list). It must not turn into an invented answer.

Step 3: Test the evidence guardrail on purpose

1. Run `persona` in `card` mode and give it no research at all.

What happens	Meaning
The skill refuses to invent users and asks you for research or interviews.	Correct. This is the guardrail working.
The skill writes a finished persona with names, ages and quotes.	Defect. Tell the observer straight away.

Step 4: Audit

Pick the audit that matches what you made. An audit only finds problems. It fixes nothing.

You made	Run	What you get
A prototype screen	`audit`, mode `craft`	A scored critique and a task list.
A prototype screen	`audit`, mode `heuristic`	A 12-point Nielsen heuristics pass. A different rubric from `craft`, so do not compare the two scores.
A prototype with real code	`audit`, mode `a11y`	A WCAG accessibility pass, criterion by criterion.
A prototype with real code	`audit`, mode `code`	A coding-standards and design-token check.

You made	Run	What you get
A build and an approved design	`audit`, mode `spec`	The build checked against the design, screen by screen.
Research or planning documents	The skill's own review checklist	These skills end with one.

1. Beside each finding write one word: **accept**, **reject** or **unsure**. Nothing is fixed until you say so.

Step 5: Review and stop

1. Read what you produced. Close the session. Fill in section 12.

A skill finished properly when all three are true: you saw a plan before any file was written; every claim is tagged evidenced, inferred or assumed, or is listed under unknowns; and you reviewed the output and said accept, reject or unsure. If one is missing, report it even if the output looks good.

10. Do you have a design system?

Ask yourself one question: do I already have a design system? This is advice, never a blocker. Claude will ask you. You can say "skip" and carry on. Anything built without a real system is labelled "placeholder tokens", so nobody mistakes placeholder values for real ones.

Your situation	What to do	What you get
A. Tokens already in code. An `index.css` with colour and spacing variables.	Run `styleguide` and ask it to audit and document what exists.	A `DESIGN.md` generated from your real tokens.
B. A brand, but no system. A brand guide PDF, a live website, or hex codes you can type.	Run `styleguide` in `forge` mode and give it your sources. If two sources disagree, it must ask which one wins.	Tokens cast from your brand, each value traced to a source.
C. A Figma library only.	Set up the Figma MCP server (section 8). Run `figma` in `import` mode, then `sync` to keep tokens aligned.	Code that follows your library.
D. Nothing yet. Most first runs land here.	Run `styleguide` in default mode.	A placeholder palette. Every screen is labelled "placeholder tokens". Come back to B later.

> **CHECK**
> Whichever you choose, ask Claude: which of these values came from my sources and which did you choose? A correct answer separates the two.

11. If something goes wrong

What you see	Likely cause	What to try
Installer says "permission denied"	The file is not marked executable	Run `chmod +x install.sh`, then the installer again.
"command not found" or "bad interpreter"	Windows line endings, or not a bash shell	Use WSL, or copy the folders by hand.
Typing / shows no UAIX Lab-Atlas skills	Claude started before the install	Quit Claude fully and reopen it. Then re-download the ZIP, run the installer again and check the count says 30.
Skill count is not 30	Partial download	Download the ZIP again and rerun the installer.
/help shows a different help screen	The built-in command takes priority	Type: Use the UAIX Lab-Atlas help skill.
A skill starts writing before it shows a plan	The plan-first step was skipped	Type: stop, show me your plan first. Then tell the observer which skill it was.
Claude asks for a password or token	It should not	Do not enter it. Stop and tell the observer.
`/mcp` shows a server as failed	Sign-in did not finish	Run `/mcp` again and repeat the browser sign-in.
Track 2: a skill uploaded but is ignored	It is switched off, or the ZIP holds loose files	Check the switch. Make the ZIP again from the folder itself.
A skill names another skill you cannot find	A stale reference	Write the name down and report it.
GitHub shows "404 Not Found"	Signed in to the wrong account	Switch to the account that was invited.

12. Tell us what happened

Fill in the feedback form your observer gives you (`onboarding/FEEDBACK-FORM.md` in the repository) right after your session, while you remember it. Honest answers help most, including "I got confused here". These are the questions:

- Which route (A, B or C), which track (1 or 2), and which operating system?
- How long from opening the link to a successful check in section 7?
- Where did you first hesitate, and why?
- Was there a step where the guide did not match your screen? Write the section and step.
- Did any skill write a file before showing you a plan? Which one?

- Did the persona test in section 9 refuse correctly?
- Which audit did you run, and how many findings did you accept, reject and mark unsure?
- Did you say yes to something you did not fully understand?
- Would you use these skills again on real work? What would stop you?

Observers: run the session with `onboarding/OBSERVER-SCRIPT.md` and log findings in `onboarding/FINDINGS-LOG.csv`. Do not help unless the tester has been stuck for 5 minutes or rule G7 triggers.

Appendix A. The 30 skills

Type `/` in Claude Code to see them. A skill with modes asks which one you want, or you can name it, for example: use audit in heuristic mode.

Phase	Skill	What it does	Needs outside your first run
Before Discover	`setup`	Creates the documentation folder, README, project AGENTS.md and an empty unknowns file.	Nothing
	`brief`	Interviews you for the product brief and records what is still unknown.	Nothing
Discover	`workshop`	Modes `plan` (agenda) and `ingest` (turns photos of a physical board into a deck, doc, CSV or FigJam board).	Figma MCP for the FigJam output
	`competitor-analysis`	Modes `discover`, `add`, `analyse`: profiles real competitors with evidence.	Web search; browser tooling for screenshots
	`ux-benchmark`	Scores 3 to 5 products 1 to 5 on one rubric, with evidence.	Web search
	`interview`	Builds the question bank, script and synthesis template.	Nothing
	`interview-analysis`	Turns transcripts into evidence-backed insights with verbatim quotes.	Nothing
	`persona`	Modes `card` (research-grounded persona cards) and `day` (a day-in-the-life walkthrough).	Nothing
	`journey`	Modes `map`, `blueprint`, `pain-points`, `solutions`, `experience`.	Nothing
	`problem-statement`	Turns research into problem statements and prioritised user stories.	Nothing
	`metrics`	Defines OKRs using HEART, CASTLE or SELECT.	Nothing
Decide	`brainstorming`	Many ideas from one problem statement, then a scored shortlist.	Nothing
	`information-architecture`	Structures content, navigation and labels, with a confidence rating per decision.	Nothing

Phase	Skill	What it does	Needs outside your first run
	`map`	Modes `objects`, `site`, `flow`, `audit`, `document`: models and diagrams the product.	Figma MCP for FigJam export
	`interrogate`	Questions a hard design decision one question at a time.	Nothing
	`decisions`	Records a decision and the rejected alternatives as a short ADR.	Nothing
Design	`prototype`	Builds a React, TypeScript, Vite and Tailwind prototype.	Node.js
	`styleguide`	Two-layer design tokens and a `DESIGN.md`. Modes default and `forge`.	Nothing
	`audit`	Modes `craft`, `heuristic`, `code`, `spec`, `a11y`. Finds problems; fixes nothing.	Nothing
	`refine`	Fixes and unifies after an audit.	Nothing
	`storybook`	Organises UI into five Atomic Design tiers with Storybook stories.	Node.js
Deliver	`usability-test-plan`	Test scenarios, a timed agenda and a moderator script, with a bias review.	Nothing
	`package`	Modes `full` and `lite`: packages the prototype for handoff.	Node.js
	`showcase`	A day-in-the-life PowerPoint or reveal.js deck.	Node.js; browser tooling
	`guide`	End-user documentation for the prototype.	Nothing
	`requirements`	Epics, features, user stories and acceptance criteria.	Jira or Azure DevOps MCP for export (optional)
Any time	`wiki`	Modes `compile` and `note`: a small linked markdown wiki of project notes.	Nothing
	`figma`	Modes `capture`, `import`, `sync`.	Figma MCP for `import` and `sync`
	`skill-manager`	Installs, removes, disables and lists skills.	Nothing
	`help`	Reads your project and recommends the next 1 to 3 skills, or shows the full map in `atlas` mode.	Nothing

Appendix B. Every link in this guide

Link	What it is
https://github.com/RPalis/ux-atlas-aixlab	The repository
https://github.com/RPalis/ux-atlas-aixlab/tree/update-readme-retire-competitors-duplicate	The branch with the work and this guide
https://github.com/RPalis/ux-atlas-aixlab/archive/refs/heads/update-readme-retire-competitors-duplicate.zip	The branch as a ZIP (needs the invited account for the private repository)
github.com/signup	Create a GitHub account
claude.com/download	Claude desktop app downloads
claude.com/pricing	Claude plans
claude.ai	Claude in the browser
code.claude.com/docs/en/quickstart	Claude Code install and first session
code.claude.com/docs/en/terminal-guide	How to open and use a terminal
code.claude.com/docs/en/mcp	Adding MCP servers to Claude Code
support.claude.com/en/articles/12512180-using-skills-in-claude	Using skills in Claude (browser)
help.figma.com Guide to the Figma MCP server	Figma MCP server setup
https://mcp.figma.com/mcp	Figma remote MCP server address (used in a command, not opened in a browser)
git-scm.com/downloads/win	Git for Windows
learn.microsoft.com/windows/wsl/install	Windows Subsystem for Linux
nodejs.org	Node.js

Appendix C. Words used in this guide

Word	Meaning
Skill	A folder of instructions, with a file called `SKILL.md`, that teaches Claude one job.
Mode	A named variant inside one skill, such as `audit` mode `a11y`.
Claude Code	Claude working in a project folder on your computer, in the desktop app's Code tab or in a terminal. It can read and write files.

Word	Meaning
Plan mode	A Claude Code setting where Claude shows a plan and waits for you before changing anything.
MCP server (connector)	A bridge that lets Claude use another tool, such as Figma, after you sign in to that tool.
Observer	The person watching your session to test this guide. They do not grade you.
Branch	A named version of the repository. The work is on `update-readme-retire-competitors-duplicate`, not on `main`.
Evidenced, inferred, assumed	The tags on every claim: backed by a source, reasoned from one, or an assumption listed openly.
Unknowns	The running list of questions nobody has answered yet. Skills write here instead of guessing.